

YANFLÈ — Guide de fine-tuning

Guide : entraîner un nouveau modèle pour la chaîne YANFLÈ

Ce guide explique comment entraîner (fine-tuner) un modèle YOLO supplémentaire pour la chaîne de détection, en prenant comme exemple concret le modèle **cagoule** / **cache-nez** (détection de dissimulation du visage), qui n'existe pas encore en pré-entraîné fiable et doit donc être entraîné nous-mêmes.

La méthode est la même pour n'importe quel futur modèle : seul le dataset change.

1. Ce dont tu as besoin

- Python 3.10+ avec `pip install ultralytics roboflow`
- Un compte gratuit sur roboflow.com + une clé API (Settings → API Keys)
- Idéalement un GPU (sinon l'entraînement tourne sur CPU, juste plus lentement)

2. Récupérer le dataset

Sur [Roboflow Universe](https://universe.roboflow.com), cherche un dataset contenant la classe **balacclava** (il en existe qui combinent plusieurs classes de sécurité : **hat**, **helmet**, **sunglasses**, **balacclava**, **face-mask**...). Ouvre le projet, onglet **Dataset**, choisis le format d'export **YOLOv8**.

Télécharge-le directement en Python avec ta clé API :

```
from roboflow import Roboflow

rf = Roboflow(api_key="TA_CLE_API")
project = rf.workspace("NOM_WORKSPACE").project("NOM_PROJET")
dataset = project.version(1).download("yolov8")
```

Cela crée un dossier avec `data.yaml`, `train/`, `valid/`, `test/`.

Attention — vérifie dans `data.yaml` la liste `names` : si le dataset a plusieurs classes (casque, gilet...) et que tu ne veux entraîner que sur la cagoule, tu peux soit garder toutes les classes (le modèle sera multi-usage), soit filtrer le dataset pour ne garder que les images/labels de la classe qui t'intéresse.

3. Lancer l'entraînement

Depuis le dossier contenant `data.yaml` :

```
yolo detect train \
  model=yolov8n.pt \
```

```
data=data.yaml \
epochs=100 \
imgsz=640 \
patience=20 \
name=balaclava_run
```

- On part de yolov8n.pt (le plus petit modèle pré-entraîné COCO), pas de zéro : c'est du fine-tuning, beaucoup plus rapide.
- epochs=100 avec patience=20 : l'entraînement s'arrête tout seul s'il n'y a plus d'amélioration après 20 epochs, pas besoin de deviner le bon nombre à l'avance.
- Sur un petit dataset (quelques milliers d'images), compte de 30 min à 2h sur un GPU grand public, plusieurs heures sur CPU.

Les résultats vont dans runs/detect/balaclava_run/ : - weights/best.pt → le fichier à utiliser (meilleur score sur le jeu de validation) - results.png, confusion_matrix.png → à regarder pour juger la qualité

4. Vérifier avant d'intégrer

Ne jamais brancher un modèle dans la chaîne sans avoir vérifié ses vrais résultats :

```
yolo detect val model=runs/detect/balaclava_run/weights/best.pt data=data.yaml
```

Regarde le mAP50 affiché. Comme repère, les modèles déjà dans la chaîne (menace, feu, bagarre) tournent autour de 0.6 à 0.9 de mAP50 selon la difficulté du sujet. En dessous de ~0.5, le modèle n'est pas assez fiable pour être utilisé tel quel — il faut plus de données ou plus d'epochs.

Teste aussi visuellement sur quelques images/vidéos qui n'étaient pas dans le dataset d'entraînement, pour repérer les faux positifs évidents (ex : confondre une écharpe avec une cagoule).

5. Intégrer le modèle dans la chaîne

1. Copie le fichier dans v1_yolo/models/, avec un nom qui suit la convention déjà en place (<sujet>-<variante_yolo>.pt), par exemple :

```
cp runs/detect/balaclava_run/weights/best.pt /home/rakine/osc/v1_yolo/models/balaclava-yol
```

2. Ajoute une entrée dans MODELS_INFO (v1_yolo/backend/main.py), avec le même format que les modèles existants :

```
{
  "id": "balaclava",
  "role": "Le Guetteur des visages",    # à adapter, garde un nom cohérent avec les autres
  "focus": "Repère un visage volontairement dissimulé (cagoule, cache-nez)",
  "file": "balaclava-yolov8n.pt",
  "conf": 0.6,    # à ajuster selon les résultats de la validation (étape 4)
}
```

Si le dataset a plusieurs classes et que tu ne veux garder que balaclava, utilise la clé "classes": [i] avec l'index correspondant dans data.yaml (voir l'exemple du modèle fight-yolov8s.pt déjà présent dans MODELS_INFO).

3. Ajoute la traduction française du label dans `v1_yolo/backend/labels_fr.py` (même dictionnaire que pour les autres classes), pour que la narration déterministe (`narrator.py`) affiche du français correct.
4. Redémarre le backend (`pkill -f "uvicorn main:app"` puis relance) : le nouveau modèle sera chargé automatiquement pour chaque caméra via `get_models_for_camera()`, sans autre changement de code.

6. Rappel important

Le fichier `narrator.py` du projet est **100% déterministe, sans IA générative** : il ne fait que décrire les faits détectés par les modèles. N'ajoute jamais de reformulation par LLM à cet endroit — c'est une règle du projet, pas un détail technique.